



Instituto Politécnico Nacional

Escuela Superior de Computo



Materia: Aplicaciones para Comunicaciones en Red

Profesor: Tanibet Pérez De los Santos Mondragón

Integrantes:

Gil Valentín Oswaldo

Olguín Hernández Johan Gael

Grupo: 6CV1 **Fecha:** 07/05/2026

Actividad: Ejercicio 3.5 Semaforos

Instrucciones:

Desarrollar una aplicación distribuida en basada en arquitectura Cliente/Servidor TCP que permita simular una arena multijugador con capacidad limitada y manejo concurrente de múltiples jugadores.

El sistema deberá implementar mecanismos de sincronización y control de concurrencia utilizando semáforos, memoria compartida y procesos o hilos concurrentes.

Descripción del Problema

Se desea implementar un sistema donde múltiples procesos cliente ("Jugadores") se conecten a un servidor central ("Arena Hub") para participar en una arena de combate.

La arena solo permitirá un máximo de 5 jugadores simultáneos. Cuando un jugador intente ingresar y la arena esté llena, deberá permanecer en estado de espera hasta que exista un espacio disponible.

El servidor será responsable de:

- aceptar conexiones TCP,

```
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen()
    print(f"[SERVIDOR] Escuchando en {HOST}:{PORT}...")
```

- administrar el acceso a la arena,

```
def handle_player(conn, addr):
    player_id = f"{addr[1]}"
    print(f"[CONEXIÓN] Intento de ingreso: {player_id}")

    try:
        # Intentar entrar a La arena (Semaforo)
        conn.sendall(b"Esperando espacio en la arena...")
        with arena_semaphore:
            print(f"[ARENA] {player_id} ha entrado a la arena.")
            conn.sendall(b"ENTRADA_CONCEDIDA")
```

Para administrar el acceso a la arena asignamos una ID al cliente y verificamos que aun haya espacio en la arena (Num máximo 5).

- mantener el estado global de los jugadores,

```
MAX_JUGADORES = 5
```

Para garantizar que efectivamente el numero máximo de jugadores sea 5 lo definimos como variable global.

- sincronizar el acceso concurrente a la información compartida.

```
# Mecanismos de Sincronización
arena_semaphore = threading.Semaphore(MAX_JUGADORES)
data_lock = threading.Lock()
```

Utilizamos semáforos y candados para mantener la sincronización de la información

```
# Inicializar estado en memoria compartida
with data_lock:
    arena_state[player_id] = {"salud": 100, "pos": (0,0), "activo": True}

while True:
    data = conn.recv(1024).decode()
    if not data or data == "QUIT":
        break
```

Cada jugador deberá mantener atributos como:

- identificador,
- salud,

```
# Inicializar estado en memoria compartida
with data_lock:
    arena_state[player_id] = {"salud": 100, "pos": (0,0), "activo": True}
```

```
# Procesar actualización de estado
try:
    hp, x, y = data.split(',')
    with data_lock:
        arena_state[player_id].update({
            "salud": int(hp),
            "pos": (int(x), int(y))
        })
```

La ID del jugador la utilizamos pero no la mostramos explícitamente, además de eso cada jugador tiene puntos de vida y una ubicación en un plano simulado.

Durante la ejecución, cada cliente simulará un combate reduciendo periódicamente su salud. Cuando la salud llegue a cero, el jugador abandonará la arena y el servidor deberá liberar automáticamente el espacio correspondiente.

```
[SERVIDOR]: Esperando espacio en la arena...
[SISTEMA] ¡Has entrado a la arena!
[ESTADO] Salud: 95% | Pos: (36, 86)
[ESTADO] Salud: 91% | Pos: (36, 85)
[ESTADO] Salud: 86% | Pos: (33, 85)
[ESTADO] Salud: 84% | Pos: (36, 85)
[ESTADO] Salud: 79% | Pos: (34, 85)
[ESTADO] Salud: 75% | Pos: (37, 86)
[ESTADO] Salud: 70% | Pos: (34, 85)
[ESTADO] Salud: 67% | Pos: (35, 84)
[ESTADO] Salud: 65% | Pos: (38, 83)
[ESTADO] Salud: 63% | Pos: (36, 83)
[ESTADO] Salud: 60% | Pos: (34, 84)
[ESTADO] Salud: 55% | Pos: (36, 84)
[ESTADO] Salud: 53% | Pos: (33, 85)
[ESTADO] Salud: 51% | Pos: (33, 85)
[ESTADO] Salud: 46% | Pos: (31, 84)
[ESTADO] Salud: 44% | Pos: (28, 85)
[ESTADO] Salud: 39% | Pos: (27, 85)
[ESTADO] Salud: 34% | Pos: (28, 85)
[ESTADO] Salud: 30% | Pos: (29, 85)
[ESTADO] Salud: 26% | Pos: (27, 86)
[ESTADO] Salud: 24% | Pos: (29, 85)
[ESTADO] Salud: 19% | Pos: (32, 84)
[ESTADO] Salud: 15% | Pos: (33, 83)
[ESTADO] Salud: 12% | Pos: (35, 83)
[ESTADO] Salud: 9% | Pos: (36, 83)
[ESTADO] Salud: 6% | Pos: (35, 82)
[ESTADO] Salud: 3% | Pos: (36, 81)
[ESTADO] Salud: 0% | Pos: (33, 81)
[SISTEMA] Has muerto. Abandonando arena...
```

Requerimientos

Servidor

El servidor deberá:

- aceptar múltiples conexiones TCP simultáneamente,

```
while True:
    conn, addr = server.accept()
    thread = threading.Thread(target=handle_player, args=(conn, addr))
    thread.start()
```

Usamos hilos para manejar las conexiones multiples.

- controlar una arena con capacidad máxima de 5 jugadores,

```
MAX_JUGADORES = 5
```

Para controlar la cantidad máxima de jugadores usamos una variable global.

- mantener el estado global de la arena en memoria compartida,

```
# Inicializar estado en memoria compartida
with data_lock:
    arena_state[player_id] = {"salud": 100, "pos": (0,0), "activo": True}
```

- sincronizar el acceso concurrente mediante semáforos o mecanismos equivalentes,

```
# Mecanismos de Sincronización
arena_semaphore = threading.Semaphore(MAX_JUGADORES)
data_lock = threading.Lock()
```

- gestionar jugadores en espera cuando la arena esté llena,

```
# Intentar entrar a la arena (Semaforo)
conn.sendall(b"Esperando espacio en la arena...")
with arena_semaphore:
    print(f"[ARENA] {player_id} ha entrado a la arena.")
    conn.sendall(b"ENTRADA_CONCEDIDA")
```

- liberar espacios automáticamente cuando un jugador muera o se desconecte,

```
# Liberar recursos y espacio cuando un jugador muere
with data_lock:
    if player_id in arena_state:
        del arena_state[player_id]
    conn.close()
    print(f"[DESCONEXION] {player_id} salió. Espacio liberado.")
```

Cliente

El cliente deberá:

- conectarse al servidor mediante TCP,

```
def start_player():
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((HOST, PORT))
```

- solicitar acceso a la arena,
- permanecer en espera si no existen espacios disponibles,

```
# Recibir estado inicial
msg = client.recv(1024).decode()
print(f"[SERVIDOR]: {msg}")

if "Esperando" in msg:
    status = client.recv(1024).decode()
    if status == "ENTRADA_CONCEDIDA":
        print("[SISTEMA] ¡Has entrado a la arena!")
```

- simular el comportamiento de un jugador dentro de la arena,

```
# Simular movimiento
pos_x += random.randint(-3, 3)
pos_y += random.randint(-1, 1)
```

Para simular el comportamiento dentro de una arena se simula el movimiento en un plano con coordenada X y Y que se mueven aleatoriamente en valores de -3 a 3 en X y de -1 a 1 en Y.

- actualizar periódicamente su estado,

```
# Enviar actualización: HP, X, Y
update = f"{salud},{pos_x},{pos_y}"
client.sendall(update.encode())
```

Actualizamos enviando los valores que tienen la posición X, Y y la Salud que tiene en ese momento.

- reducir su salud de manera automática cada cierto intervalo de tiempo,

```
# Simular combate: reducción aleatoria de salud
salud -= random.randint(2, 5)
if salud < 0: salud = 0
```

```
time.sleep(5)
```

Para simular la pérdida de salud, se baja en intervalos de 5 segundos y con valores aleatorios entre 2 y 5.

- notificar al servidor cuando abandone la arena.

Cuando la vida del jugador es menor que cero en el ciclo while se imprime un mensaje y se notifica al servidor.

```
print("[SISTEMA] Has muerto. Abandonando arena...")
client.sendall(b"QUIT")
```